

Katana Payment Gateway — Integration Guide

UPI Intent · P2P · Payout · Version 1.1 · 8 September 2026

COPYRIGHT

©2026 Katana. All rights reserved.

The material contained in this guide is copyrighted and owned by Katana together with any other intellectual property in such material. No part of this guide may be copied, republished, performed in public, broadcast, uploaded, transmitted, distributed, modified, or dealt with in any manner at all, without the prior written permission of Katana, and only in such a way that the source and intellectual property rights are acknowledged.

To the maximum extent permitted by law, Katana shall not be liable to any person or organization, in any manner whatsoever, from the use, construction or interpretation of, or the reliance upon, all or any of the information or materials contained in this guide.

The information in this material is subject to change without any prior notice. Katana assumes no responsibility for any errors.

VERSION CONTROL TABLE

Date Updated	Version	Description of Changes	Author	Approved By
8 Sep 2026	1.0	Initial version — UPI Intent, P2P and Payout	Katana Team	Katana Team
8 Sep 2026	1.1	Corrected status vocabulary, callback delivery policy and signing schemes; added order lifetime, held orders, reconciliation and go-live sections	Katana Team	Katana Team

Three things that will cost you money if you skip them.

1. Your `txnid` must be **globally unique across the Katana platform**, not just unique to you. Prefix it with your merchant code (§5.1).
2. `Expired` and `Failed` are **not final**. A payment that goes through afterwards still succeeds, and a second callback with `Captured` follows — your handler must accept it, not discard it as a duplicate (§7.3).
3. Katana gives up on a callback after 6 failed attempts. The status enquiry is the authoritative record, and a daily reconciliation sweep is mandatory (§10.2).

CONTENTS

1. ABOUT THIS GUIDE

- 1.1. Objectives and target audience
- 1.2. Integration directions
- 1.3. Status at a glance

2. INTRODUCTION

- 2.1. Pre-Requisite
- 2.2. Connecting to the Payment Gateway
- 2.3. Business lines covered
- 2.4. Test environment

3. INCEPTIVE FRAUD PREVENTION

- 3.1. Fraud analysts

4. PAYMENT PROCESS

- 4.1. Redirecting customers to the Katana Gateway (step 1)
- 4.2. UPI Intent deeplink flow
- 4.3. Order lifetime
- 4.4. Orders held for review
- 4.5. The browser return

5. INTEGRATION APPROACH

- 5.1. Request Format
- 5.2. Amount format
- 5.3. Validating the response
- 5.4. P2P collection
- 5.5. Payout
- 5.6. Limits

6. GENERATING A SECURE HASH

- 6.1. SHA256 signature
- 6.2. Secret key
- 6.3. Code integration examples

7. STATUS ENQUIRY

- 7.1. Status Enquiry Request
- 7.2. Status Enquiry Response
- 7.3. Expiry and late credits

8. MERCHANT CALLBACKS

- 8.1. Call-back Response
- 8.2. Merchant-Response
- 8.3. Delivery and retries

9. GATEWAY OPTIONS AND RESPONSE

- 9.1. Response Code for Valid Transaction
- 9.2. Response Code for Invalid Transaction
- 9.3. Payment method codes
- 9.4. Supported Currency and Currency Code

10. GOING LIVE AND DAY-2 OPERATIONS

- 10.1. Go-live checklist
- 10.2. Daily reconciliation
- 10.3. Settlement
- 10.4. Refunds and disputes

11. SUPPORT AND ESCALATION

1. ABOUT THIS GUIDE

1.1. Objectives and target audience

This guide provides detailed information on how to connect to and use the Katana Payment Gateway by integrating it into a merchant website or application. It covers the steps in the payment process and the information that must be passed from merchant web servers to Katana, to enable Katana to process payments.

It is written for the merchant's technical team. Additional gateway integration options are also described.

Items marked **DECISION** are commercial or operational values Katana has not yet fixed. They are shown as open rather than filled with a plausible number — ask your account manager for the current value before relying on one.

1.2. Integration directions

Katana onboards a counterparty in one of two directions. Which applies is agreed at onboarding, and a single counterparty may use both.

	Direction A — Katana consumes	Direction B — Katana provides
Who issues credentials	The merchant / TSP issues to Katana	Katana issues to the merchant
Who calls whom	Katana calls their API	They call Katana's API
Who receives callbacks	Katana receives theirs	They receive Katana's (§8)
Typical case	Merchant already holds a Bank API through a TSP	Merchant wants a gateway and has nothing of their own

Sections 2–11 document Direction B, where Katana issues the credentials. Direction A uses the same field vocabulary and hashing rules in reverse: the counterparty supplies the base URL, Pay ID / MID, client ID, secret, signing scheme **with a worked example**, endpoint paths and their callback specification; Katana signs outbound calls with their secret and receives their notifications on a callback URL issued at onboarding.

1.3. Status at a glance

Business line	Merchant-callable API	What you can do today
UPI Intent (pay-in)	LIVE	Full self-service integration — §4 to §9.
P2P	TO BUILD	Collections are recorded and matched by Katana operators; outcomes reach you through account-manager reporting, not an API.
Payout	LIVE	Register beneficiaries, send payouts and receive signed status callbacks — §5.5.

What TO BUILD means. The engine, its controls and its data model are live and in daily operational use inside Katana. What does not yet exist is the signed, merchant-callable endpoint. The contract in §5.4 is the specification that endpoint will be built to — stable enough to design against, and Katana will tell you before they change. The lead time is integration and testing, not building the engine.

2. INTRODUCTION

The Katana Payment Gateway is a secured payment orchestrator. You redirect customers from your website, e-commerce or m-commerce platform to make a payment using UPI, or you take the UPI Intent deeplink and present it inside your own payment page. Katana collects the payment in a secured manner and processes the transaction.

After the payment is complete, the customer is returned to your website and you receive a real-time notification of the payment, including the details of the transaction.

2.1. Pre-Requisite

- Read this guide in full before starting. The integration is straightforward for anyone with a technical background.
- A Katana merchant account, with a **Key** and **Salt** issued to it (§6.2).
- Confirmation of **which signing scheme** your Key was issued with (§6.1). Signing with the wrong one fails every request.
- A server able to POST JSON and compute an HMAC/SHA-256 or SHA-512 hash. No SDK is required and any language works.
- A **collection UPI ID** configured on your Katana account, so payments reach you and not a default (§5.1).
- An **https** callback endpoint that is publicly resolvable (§8).

2.2. Connecting to the Payment Gateway

Connecting to the Katana Payment Gateway requires adding Katana as a payment method on your checkout or payment page. When your customer selects it, your **server** creates a signed order with Katana and receives back a payment URL and a set of UPI deeplinks. You then either redirect the customer to the Katana hosted payment page, or present the deeplink or QR inside your own page.

Order creation is server-to-server. The endpoint returns a JSON body, not a browser redirect — a form posted directly from the customer's browser would land them on raw JSON. Create the order from your server, then send the customer to the `pay_url` you get back.

The Salt is used **only** on your server, to compute the `hash` you send and to verify the `HASH` on callbacks. It must never appear in browser code, a mobile application binary, or a repository.

A simplified explanation of the transaction flow:

1. The customer is ready to pay on your website and selects the Katana payment option.

2. Your server calls the Katana order API with the transaction details and a hash, and receives a `pay_url`, a UPI Intent deeplink and a QR payload.
3. The customer either lands on the Katana payment page, or taps the deeplink in your own page and is taken into their UPI application.
4. The customer authorises the payment in their UPI app.
5. Katana receives confirmation of the credit and marks the order.
6. Katana POSTs a signed callback to your server, and the customer is redirected back to your return URL.

2.3. Business lines covered

Line	What it does	Status
UPI Intent (pay-in)	Collection by UPI deeplink or QR, hosted or embedded.	LIVE
P2P	Collection recorded against a trader and matched on UTR, with limit and duplicate controls.	TO BUILD
Payout	Disbursement to an approved beneficiary by IMPS, NEFT, RTGS or UPI.	LIVE

Status tags are load-bearing. UPI Intent and Payout are callable by a merchant server today. See §1.3 for what is available on the other two lines in the meantime.

2.4. Test mode

There is no separate sandbox host. Every merchant has a **test** and a **live** Key + Salt on the same base URL, and the Key that signs an order decides its mode. A test order never reaches a bank and no money moves.

Item	Test	Live
Base URL	<code>https://katanapay.co</code> for both	
Key prefix	<code>mk_test_</code>	<code>mk_live_</code> (keys issued before test mode, <code>mk_</code> + hex, are live)
Payee	A sandbox UPI ID	The merchant's settlement UPI ID
Callbacks	Signed with the test Salt, carry <code>"LIVEMODE": "false"</code>	Signed with the live Salt
Ledger, statements, settlement	Never included	Included

Forced-outcome amounts	On, plus Simulate success / failure buttons on the hosted pay page	Off
Available	From the first day	After live mode is activated for the merchant (checklist + Katana approval); before that a live Key gets <code>403</code> <code>LIVE_MODE_NOT_ACTIVATED</code>

On a test order the paise value of the amount selects the result, so every branch can be tested without a real payer:

Send amount	Order becomes	Use it to test
<code>100.99</code>	<code>SUCCESS</code> after ~8 s	Happy path, callback verification, fulfilment
<code>100.13</code>	<code>FAILED</code>	Decline handling
<code>100.11</code>	<code>EXPIRED</code>	Expiry and re-check handling (\$7.3)
<code>100.00</code>	stays <code>PENDING</code>	Reconciliation sweep and long-poll

These hooks exist **only** for test orders. A live order changes state only on a real confirmed credit, and an amount ending in `.99` behaves exactly like any other. Never rely on them with live credentials.

3. INCEPTIVE FRAUD PREVENTION

No one knows your business and your customers better than you do, and the same applies to your fraud screening.

Anti-fraud measures combine several tests to work out the likelihood of a transaction being fraudulent:

- IP geographical location
- Buying pattern
- Purchase history
- Black / whitelists
- SQL injection checks
- Proper integration review
- Manual review

Katana offers all of these tests, but some will have more relevance to your business than others. The way you would manually score the likelihood of fraud may be more than the cumulative effect of a few tests added together.

CONTROLS KATANA ENFORCES ON EVERY TRANSACTION

Control	Effect
Signature verification	An order whose hash does not verify is rejected <code>401</code> and never created.
Blocked-account check	A suspended merchant is refused <code>403</code> .
Amount validation	A missing, zero or negative amount is refused <code>400</code> .
Final-status lock	An order that reached <code>SUCCESS</code> or <code>FAILED</code> cannot be changed by a later message; a conflicting update is rejected <code>409</code> .
Duplicate-UTR blocking	A bank reference settles exactly one pay-in.
High-value hold	Large amounts are held for operator review rather than resolved automatically (§4.4).
Callback egress guard	Katana refuses to POST a callback to a private, loopback or link-local address.

Each control above is enforced server-side on every request; none can be disabled per merchant.

3.1. Fraud analysts

Katana fraud analysts work with you to define the most appropriate screening techniques for your business. This may be a combination of standard tests, custom tests specific to your business, and can also include additional data integration, which could take the form of historical fraud or suspicious transaction data.

4. PAYMENT PROCESS

4.1. Redirecting customers to the Katana Gateway (step 1)

When a customer is on the online checkout or payment page on your website, they should be presented with a **Pay** button.

HOW TO REDIRECT THE CUSTOMER

- When the customer selects the Katana option, your **server** posts the transaction details in Table 2 to the Katana order endpoint.
- Katana returns a `pay_url`. Send the customer's browser to it.
- Obtain a secure session identifier on your side before redirecting, so you can tie the returning customer back to the pending order.

TIPS FOR IMPROVING THE CUSTOMER EXPERIENCE

- Any customer parameters you pass — name, email, phone — pre-populate the relevant fields on the Katana payment page, making it easier for the customer to complete.
- Redirect the customer in the **same browser** to maximise conversion.
- On a mobile device, prefer the UPI Intent deeplink (§4.2) — it removes a page load and drops the customer straight into their UPI application.

4.2. UPI Intent deeplink flow

Where you would rather keep the customer inside your own page, use the deeplink and QR payload returned with the order instead of redirecting.

Field in the response	Use
<code>upi_intent</code>	A <code>upi://pay?...</code> deeplink. Bind it to your Pay button on mobile.
<code>deeplinks</code>	Per-application variants, for showing named app buttons.
<code>qr_payload</code>	Render as a QR code for desktop checkout or a counter display.
<code>pay_url</code>	The Katana hosted page, if you prefer not to build a payment screen at all.

Host-to-host or redirect. `checkout` in the response says which you got: `H2H` when the UPI link and QR are there for your own page, `REDIRECT` when they are not. Which one depends on the payment account behind your channel; ask Katana for host-to-host if your integration needs it. **Merchants paid on a gateway's own page** (`REDIRECT`) get no deeplink or QR: `upi_intent` , `deeplinks` and `qr_payload` are `null` , and the response adds `gateway_url` (that page). Some host-to-host orders carry `gateway_url` too: the UPI link is then also there, and either way works. Send the customer to `pay_url` , which hands over to the gateway, or straight to `gateway_url` . The gateway's own limits apply, e.g. a minimum amount per payment (a refused order returns `502` with the reason).

The customer usually does not come back. On UPI the customer approves inside their banking application and frequently never returns to the browser, so the redirect alone must never be treated as confirmation. Rely on the callback (§8) and the status enquiry (§7).

4.3. Order lifetime

Validity window	15 minutes from order creation: the customer's time to pay
At the end of it	An order still <code>PENDING</code> becomes <code>EXPIRED</code> (<code>003</code>) and the callback fires

Confirmation period	An order paid on a payment processor's page may be kept <code>PENDING</code> for a further period after the 15 minutes while the payment is confirmed. The customer cannot start a new payment in that period. <code>EXPIRED</code> and its callback come at the end of it
Exception	Orders held for review (§4.4) do not expire — they wait for a Katana operator
Late payment	A credit that lands after expiry still counts — see §7.3

Do not display a `pay_url`, deeplink or QR for longer than the validity window, and never email one — it will usually have expired before it is opened. If the customer has not paid, create a **new order with a new `txnid`** rather than re-showing the old one.

Expiry is evaluated when the order is next read or swept, not by a precise timer, so an order may report `PENDING` slightly past the 15-minute mark. Do not time an order out yourself at 15 minutes: wait for the callback, or read the status (§7).

4.4. Orders held for review

Some orders are parked for a Katana operator instead of resolving automatically:

Trigger	Effect on your order
Amount at or above ₹50,000	Stays <code>PENDING</code> . Does not expire at 15 minutes. Resolves only when an operator confirms or rejects it.
The payer uploaded a payment proof on the Katana page	Same. The status enquiry returns <code>proof_submitted: true</code> .

When the operator decides, the normal terminal callback fires.

A held order has no documented upper bound on how long it stays `PENDING`. Your reconciliation sweep must be able to carry an order across days, and your checkout must not assume every order reaches a terminal state inside the validity window. If you cancel a customer's order after 30 minutes of `PENDING`, you will cancel legitimate high-value payments that are merely awaiting review.

DECISION The ₹50,000 threshold is deployment-wide, not per merchant, and there is no published review SLA. Confirm both with your account manager before designing a timeout.

4.5. The browser return

When an order reaches a terminal state on the hosted page, the customer's browser is redirected to your `return_url` with three query parameters appended:

https://yoursite.com/return?order_id=ESN78452&status=SUCCESS&rrn=123456789101

These parameters are unsigned and attacker-controlled. Treat them as display only. Anyone can type that URL into a browser. Use them to decide which "thank you" screen to render, and nothing else. Never create, complete, or fulfil an order on the strength of the browser return — confirm server-side with the callback (§8) or the status enquiry (§7) first.

5. INTEGRATION APPROACH

5.1. Request Format

REQUEST URLS

Purpose	Environment	URL	Method
Create order	Production	https://katanapay.co/api/v1/katana-pay/order	POST
Hosted payment page	Production	https://katanapay.co/pay/{katana_order_id}	GET
Status enquiry	Production	https://katanapay.co/api/pay-status/{katana_order_id}	GET

See §2.4 for test mode. {katana_order_id} is the order.id UUID returned on order creation — **not** your txnid . Sending your own reference here returns 404 .

TABLE 2 — KATANA PAYMENT REQUEST PARAMETERS

Field name	Description	Required	Type	Len min	Len max	Example
Merchant details						
key	Merchant Key issued by Katana	YES	AN	8	64	mk_9f2c...
txnid	Your reference. Must be unique across the whole Katana platform , not just your account. Idempotency key.	YES	AN	1	60	UK108-ESN78452
hash	Value generated by your signing scheme (§6)	YES	AN	64	128	7995156ce4c4...
return_url	URL on your site the customer returns to. http(s) only.	NO	CH	5	1024	https://yoursite.com/return

<code>notify_url</code>	URL Katana POSTs the status callback to. http(s) only, publicly resolvable.	NO	CH	5	1024	https://yoursite.com/callback
Customer details						
<code>firstname</code>	Customer name. Part of the signature under the SHA-512 scheme.	NO	CH	1	150	David
<code>email</code>	Customer email. Part of the signature under both schemes.	NO	CH	6	120	david@example.com
<code>phone</code>	Customer phone, without country code	NO	NU	10	10	7417456565
<code>customer_vpa</code>	Payer UPI ID, where known	NO	AN	3	120	david@bank
Payment details						
<code>amount</code>	Total sale amount in rupees (§5.2)	YES	NU	1	12	100
<code>currency</code>	Currency. Defaults to <code>INR</code> . Only <code>INR</code> is supported (§9.4).	NO	CH	3	3	INR
<code>mode</code>	<code>INTENT</code> or <code>QR</code> . Defaults to <code>QR</code> .	NO	CH	2	6	INTENT
<code>receiver_vpa</code>	Collect into a specific UPI ID instead of your account default.	NO	AN	3	120	you@bank
<code>receiver_vpas</code>	Up to 30 collection UPI IDs; the first is used, the rest are backups.	NO	array	1	30	["a@bank","b@bank"]
Order level details						
<code>productinfo</code>	Description of the order. Part of the signature under both schemes.	NO	CH	1	1024	Order 1001

* Abbreviations: NU – numeric, CH – character, AN – alphanumeric. The Len columns are **string lengths**, not value limits — see §5.6 for transaction limits.

txnid must be globally unique — prefix it. Katana's idempotency is keyed on `txnid` alone, not on `txnid` per merchant. If you send one that already exists on the platform, you receive **that** order back with `reused: true` and HTTP 200 — not a new one. Prefix every reference with your merchant code (`UK108-...`) or use a UUID. A bare sequence such as `ORDER-1001` will eventually collide.

Always check `reused` and `order.amount` on the response. If `reused` is `true` and `order.amount` is not the amount you sent, you have collided with an existing order: abandon that reference, do **not** send the customer to that `pay_url`, and alert your team.

If you send no receiver, Katana uses the collection UPI ID configured on your account. Confirm with your account manager that it is set **before** your first live order. An order created with no receiver on either side still returns a valid-looking QR and deeplink — pointing at a default payee. On your first test payment, verify that the payee shown in the customer's UPI app is yours.

TABLE 3 — PARAMETERS RETURNED ON THE ORDER RESPONSE

Field name	Description	Example value
<code>verified</code>	Signature verification result	<code>true</code>
<code>merchant</code>	Merchant code the order was booked to	<code>UK-108</code>
<code>reused</code>	Check this. <code>true</code> means the <code>txnid</code> already existed and this is the pre-existing order.	<code>false</code>
<code>order.id</code>	Katana order identifier (UUID). Addresses the payment page and status enquiry.	<code>8d20c0b6-...</code>
<code>order.order_id</code>	Your reference, echoed back	<code>UK108-ESN78452</code>
<code>order.amount</code>	Amount booked. Compare with what you sent when <code>reused</code> is <code>true</code> .	<code>100</code>
<code>order.status</code>	Initial status	<code>PENDING</code>
<code>checkout</code>	<code>H2H</code> (host-to-host): the UPI link and QR below are set, for your own page. <code>REDIRECT</code> : they are <code>null</code> ; send the customer to <code>pay_url</code> .	<code>H2H</code>

live_checkout	Test orders only: what your live orders will get (H2H , REDIRECT , or null while no payment account is set up). A test order always carries the sandbox UPI link.	REDIRECT
pay_url	Hosted payment page to redirect the customer to	https://katanapay.co/pay/8d20c0b6-...
upi_intent	UPI Intent deeplink	upi://pay?pa=...&am=100.00
deeplinks	Per-application deeplink variants	{ "upi": "...", ... }
qr_payload	Payload to render as a QR code	upi://pay?...

Additional fields may appear on the response. Ignore any you do not recognise.

```
POST https://katanapay.co/api/v1/katana-pay/order
Content-Type: application/json
```

```
{ "key": "mk_9f2c...",
  "txnid": "UK108-ESN78452",
  "amount": "100",
  "productinfo": "Order 1001",
  "firstname": "David",
  "email": "david@example.com",
  "phone": "7417456565",
  "currency": "INR",
  "mode": "INTENT",
  "hash": "<see section 6>",
  "return_url": "https://yoursite.com/return",
  "notify_url": "https://yoursite.com/callback" }
```

```
HTTP 201 Created (HTTP 200 with "reused": true if the txnid already existed)
```

```
{ "verified": true,
  "merchant": "UK-108",
  "reused": false,
  "order": { "id": "8d20c0b6-...", "order_id": "UK108-ESN78452", "amount": 100, "status": "PENDING" },
  "checkout": "H2H",
  "pay_url": "https://katanapay.co/pay/8d20c0b6-...",
  "upi_intent": "upi://pay?pa=...&am=100.00&tr=UK108-ESN78452",
  "deeplinks": { "upi": "upi://pay?..." },
  "qr_payload": "upi://pay?..." }
```

5.2. Amount format

The amount of the transaction is expressed in **rupees** only. The amount must not contain thousands separators or currency symbols. The value cannot be negative or zero.

For example, INR 12.50 is expressed as `12.50` ; INR 100 is expressed as `100` .

Format the amount once and reuse the exact string. The value you sign must be byte-identical to the value you send. Build the string first, hash that string, then send that same string. A JSON library that re-serialises `"100.00"` as `100` after you signed it will produce `401 signature mismatch` on every request.

The same four inputs signed with two different amount strings produce two completely different hashes:

```
Key mk_test · Salt s3cr3t · txnid UK108-ESN78452 · productinfo "Order 1001" · email
david@example.com

amount = "100"      -> message UK108-ESN78452|100|Order 1001|david@example.com
amount = "100.00"   -> message UK108-ESN78452|100.00|Order 1001|david@example.com
                        ^^ different string, different hash
```

The callback returns the amount in a different format. Katana normalises it: ₹100.50 comes back as `"100.5"` and ₹100.00 as `"100"` . Compare amounts **numerically**, never as strings — see §8.1.

5.3. Validating the response

Katana recommends that you validate the transaction details in the response.

IF THE INTEGRATION APPROACH IS "HOSTED PAYMENT PAGE"

1. Create a pending transaction or order for a fixed amount on your website.
2. Redirect the customer to the Katana payment page, where they complete the transaction.
3. Katana posts the transaction confirmation to your callback URL, including the amount.
4. Validate the parameters received by recalculating the SHA-256 signature (§6.1 B). If it matches, compare `AMOUNT` **numerically** against your pending order (§8.1), and compare `ORDER_ID` .
5. Before delivering, require `STATUS = "Captured"` and `RESPONSE_CODE = "000"` .
6. Perform a **double verification** with the status enquiry (§7) — note it uses a **different status vocabulary**; deliver on `status = "SUCCESS"` .
7. Compare status values case-insensitively.
8. Once validated, process the transaction — for example, by dispatching the goods ordered.

IF THE INTEGRATION APPROACH IS "UPI INTENT DEEPLINKS"

1. Create a pending transaction or order for a fixed amount on your website using the order API.
2. Take `upi_intent` (or `qr_payload`) from the response and present it in your own payment page.
3. Katana posts the transaction confirmation to your callback URL.

4. Validate the signature, amount and order reference exactly as above, then double-verify with §7, then deliver.

Please note: the implementations above are mandatory for all merchants. In the case of any discrepancy at the merchant's end, the merchant will be liable for any resulting losses. Katana will review the merchant system before go-live to confirm this — §10.1 lists what the review covers.

5.4. P2P collection TO BUILD

P2P records a payment received against a registered trader and matches it on the bank reference (UTR), applying per-transaction, daily-amount and daily-count limits. **The matching engine and its controls are live inside Katana; the merchant-facing signed endpoint below is specified, not yet released.** See §1.3 for what is available today.

Purpose	Method	Proposed URL
Record a collection	POST	<code>https://katanapay.co/api/v1/p2p/collect</code>
Collection status	GET	<code>https://katanapay.co/api/v1/p2p/collect/{id}</code>

```
{ "key": "mk_9f2c...", "txnid": "UK108-P2P-5501", "trader_id": "...",  
  "amount": "2500", "utr": "123456789012",  
  "vpa": "payer@bank", "expected_amount": "2500",  
  "hash": "<see section 6>" }
```

OUTCOMES

Match result	Status	Meaning
CORRECT	SUCCESS	The UTR is new and the amount matches. Credited.
WRONG_AMOUNT	MANUAL_REVIEW	Amount differs from <code>expected_amount</code> . Held for review.
OVER_PER_TXN_LIMIT	MANUAL_REVIEW	Above the trader's per-transaction limit.
OVER_DAILY_AMOUNT	MANUAL_REVIEW	Above the trader's daily amount limit.
OVER_DAILY_COUNT	MANUAL_REVIEW	Above the trader's daily transaction count.
DUPLICATE	FAILED	That UTR already settled a collection. Rejected <code>409</code> .

Limits are configured per trader. An inactive trader is refused `403` before any limit is evaluated, and an unknown trader returns `404` . A rejected collection is still written to the ledger, so the rejection is auditable.

Do not treat 409 DUPLICATE as success. The duplicate check is platform-wide: a 409 means that UTR is already recorded *somewhere*, which may be against a different trader entirely, not necessarily your own earlier retry. Investigate before crediting anyone.

Omitting expected_amount disables amount matching — every collection then evaluates as `CORRECT`. Always send it.

5.5. Payout LIVE

Payout sends money to a beneficiary you registered and Katana approved in advance, by IMPS, NEFT, RTGS or UPI. Requests are signed with the **same Key + Salt** as pay-ins; the Key decides test or live.

Purpose	Method	URL	Signed string
Register a beneficiary	POST	<code>https://katanapay.co/api/v1/payouts/beneficiaries</code>	<code>beneficiary_ref name account_number ifsc upi_id</code>
Create a payout	POST	<code>https://katanapay.co/api/v1/payouts/create</code>	<code>txnid amount beneficiary rail purpose</code>
Payout status	POST	<code>https://katanapay.co/api/v1/payouts/status</code>	<code>txnid (or payout_id)</code>

The hash uses your account's scheme (§6.1) over **these** fields instead of the pay-in fields:

```
Scheme 1 hash = SHA512( key|f1|f2|...|salt )           // lowercase hex
Scheme 2 hash = HMAC_SHA256( key = KEY + SALT , f1|f2|... ) // lowercase hex
```

A field you leave out is signed as an **empty string in its position**. `beneficiary` is whichever of `beneficiary_ref` or `beneficiary_id` you send. `amount` is a string (at most two decimals) and is signed **exactly as sent**.

Key mk_test · Salt s3cr3t · Scheme 1

```
txnid=P0-9001 amount=1500.00 beneficiary_ref=vendor-7 rail=IMPS purpose=vendor settlement
seq = mk_test|P0-9001|1500.00|vendor-7|IMPS|vendor settlement|s3cr3t
hash = SHA512(seq) -> lowercase hex
```

POST /api/v1/payouts/create

```
{ "key": "mk_test", "txnid": "P0-9001", "amount": "1500.00",
  "beneficiary_ref": "vendor-7", "rail": "IMPS", "purpose": "vendor settlement",
  "notify_url": "https://yoursite.com/payout-callback", "hash": "..." }
```

```
201 { "payout": { "payout_id": "P0-3FA2C19B7D10", "txnid": "P0-9001", "status": "PROCESSING",
                  "terminal": false, "amount": "1500.00", "rail": "IMPS", "utr": null, ... },
      "reused": false, "approval_required": false }
```

BENEFICIARY LIFECYCLE

```
{ "key": "mk_test", "beneficiary_ref": "vendor-7", "name": "Asha Traders",
  "account_number": "001122334455", "ifsc": "HDFC0001234", "hash": "..." }
```

Send `account_number` + `ifsc` , or `upi_id` , or both. A beneficiary is created `PENDING` and can't be paid until someone at Katana approves it. Re-sending the same `beneficiary_ref` with the same details returns it with its current status, so the same call is your status check. A `beneficiary_ref` can't be reused for different details (409) — register a changed account under a new ref. Build registration into onboarding, not into your payment path.

STATUSES

Status	Meaning
PROCESSING	Accepted; on its way to the bank.
ON_HOLD	Waiting for approval (payouts of ₹50,000 and above) or a review.
SUCCESS	Paid. <code>utr</code> carries the bank reference.
FAILED	Not paid. <code>failure_reason</code> says why. Nothing left your account.
REJECTED	Refused by the approver, or cancelled before sending.
REVERSED	Paid, then returned by the beneficiary's bank.

STATUS CALLBACK

When a payout reaches `SUCCESS` , `FAILED` , `REJECTED` or `REVERSED` , Katana POSTs a JSON callback to the payout's `notify_url` (else your webhook URL), once per status:

```
{ "EVENT": "payout.status", "PAYOUT_ID": "P0-3FA2C19B7D10", "ORDER_ID": "P0-9001",
  "AMOUNT": "1500.00", "CURRENCY_CODE": "356", "STATUS": "SUCCESS", "RAIL": "IMPS",
  "UTR": "327012345470", "FAILURE_REASON": "", "RESPONSE_DATE_TIME": "2026-09-17T10:41:02.114Z",
  "HASH": "..." }
```

Verify `HASH` exactly as for a pay-in callback (§6.1 B) — the same verifier works. `ORDER_ID` is your `txnid` ; `EVENT` tells a payout callback from a pay-in one. Test payouts also carry `LIVEMODE=false` and are signed with your test Salt. Return `2xx` ; failed deliveries are retried after 1m, 5m, 15m, 1h, 6h and 24h.

CONTROLS

Control	Rule
Approved beneficiary	Must exist and belong to your account (else <code>404</code>) and be <code>APPROVED</code> (else <code>409</code>).
Maker ≠ checker	The Key that registered a beneficiary can't approve it; a person at Katana does.
Balance	The payout account must cover the amount, else <code>409</code> .
Approval on high value	Payouts of ₹50,000 and above return <code>ON_HOLD</code> — queued, not paid.
Rails	IMPS up to ₹5,00,000; RTGS from ₹2,00,000; UPI needs a <code>upi_id</code> ; IMPS/NEFT/RTGS need account + IFSC. Default: IMPS for a bank beneficiary, UPI for a UPI-only one.
Test mode	A test Key pays out only through the payout provider's sandbox and never moves real money. A Key only sees payouts of its own mode.

A timeout is not a failure. `/api/v1/payouts/create` is idempotent on `txnid` : retry with the **same** `txnid` (you get the first payout back, `reused: true`) or ask `/api/v1/payouts/status` . A new `txnid` is a second payout, and a second payout can't be recalled. The same `txnid` with a different amount or beneficiary is refused with `409` .

5.6. Limits DECISION

Limit	Value
Minimum transaction	DECISION — the API rejects zero and negative amounts
Maximum transaction	DECISION — orders $\geq$ ₹50,000 are held for review (§4.4)
Daily / monthly volume	DECISION — per your commercial agreement
Order-creation rate	DECISION — no rate limit is currently enforced

Your code creates a secure hash over the transaction request data. Katana creates another secure hash and sends it back to you in every callback, so both directions are verifiable.

Which scheme do I use? Your account is issued with **one** of the two below, and it is shown next to your Key in your portal. Signing with the wrong one produces `401 signature mismatch` on every request. Every Key issued or regenerated now uses **Scheme 2** (HMAC-SHA256); **Scheme 1** applies only to Keys issued before that and keeps working until they are regenerated. Confirm before you write any code.

[illegible]

There are exactly **eleven** | characters between `email` and `salt` — five for the unused user-defined fields and five reserved, each contributing one empty segment.

```
Key mk_test · Salt s3cr3t

txnid      = UK108-ESN78452
amount     = 100
productinfo = Order 1001
firstname  = David
email      = david@example.com

seq  = mk_test|UK108-ESN78452|100|Order 1001|David|david@example.com|||||||s3cr3t
hash = SHA512(seq) -> lowercase hex
```

```
message = txnid + "|" + amount + "|" + productinfo + "|" + email
hash    = HMAC_SHA256( key = KEY + SALT , message )      // lowercase hex, 64 characters
```

```
Key mk_test · Salt s3cr3t
```

```
message = UK108-ESN78452|100|Order 1001|david@example.com  
HMAC key = mk_tests3cr3t  
hash     = HMAC_SHA256(HMAC key, message) -> lowercase hex
```

Under either scheme, a field you omit is signed as an **empty string in its position** — the separators remain. Note that `firstname` is signed under Scheme 1 and not under Scheme 2.

Self-check before your first call. Print your signing string with the separators visible and count them. `signature mismatch` is almost always a wrong scheme, a reformatted amount (\$5.2), or a wrong pipe count — very rarely a wrong Salt. See §9.2 for a full debugging order.

B. VERIFYING A RESPONSE OR CALLBACK FROM KATANA

The callback hash is computed over **sorted name–value pairs**, which is a different construction from the request hash above. To verify:

1. Take every field received **except** `HASH`.
2. Sort the field names in **ascending byte order**.
3. Join them as `NAME=value` with a **tilde** `~` between pairs.
4. Append your **Salt** directly to the end of that string, with no separator.
5. Apply SHA-256 and convert the result to **UPPERCASE** hexadecimal.
6. Compare with the `HASH` field, in constant time.

WORKED EXAMPLE 1 — A CAPTURED PAYMENT

```
{ "AMOUNT": "100",  
  "CURRENCY_CODE": "356",  
  "ORDER_ID": "UK108-ESN78452",  
  "PAY_ID": "9323643809301818",  
  "RESPONSE_CODE": "000",  
  "RESPONSE_DATE_TIME": "2026-09-08T11:35:42.991Z",  
  "RRN": "123456789101",  
  "STATUS": "Captured",  
  "TXN_ID": "4694e5ce-b8cd-475a-82b9-eec7bb846367",  
  "HASH": "8EAC6B6D..." }
```

String to hash (shown wrapped; the real string has no line breaks):

```
AMOUNT=100~CURRENCY_CODE=356~ORDER_ID=UK108-ESN78452~PAY_ID=9323643809301818~  
RESPONSE_CODE=000~RESPONSE_DATE_TIME=2026-09-08T11:35:42.991Z~RRN=123456789101~  
STATUS=Captured~TXN_ID=4694e5ce-b8cd-475a-82b9-eec7bb846367<YOUR_SALT>
```

WORKED EXAMPLE 2 — A FAILED PAYMENT, WITH AN EMPTY FIELD

A failed order carries an **empty** `RRN`. This is the case that breaks most verifiers, and it is not hypothetical — it is what every failed order looks like.

```
{ "AMOUNT": "100", "CURRENCY_CODE": "356", "ORDER_ID": "UK108-ESN78452",  
  "PAY_ID": "9323643809301818", "RESPONSE_CODE": "004",  
  "RESPONSE_DATE_TIME": "2026-09-08T11:41:02.114Z",  
  "RRN": "", "STATUS": "Failed",  
  "TXN_ID": "4694e5ce-b8cd-475a-82b9-eec7bb846367", "HASH": "..." }
```

String to hash — note `RRN=` is **present with nothing after the equals sign**:

```
AMOUNT=100~CURRENCY_CODE=356~ORDER_ID=UK108-ESN78452~PAY_ID=9323643809301818~  
RESPONSE_CODE=004~RESPONSE_DATE_TIME=2026-09-08T11:41:02.114Z~RRN=~  
STATUS=Failed~TXN_ID=4694e5ce-b8cd-475a-82b9-eec7bb846367<YOUR_SALT>
```

Empty and null values still take part. Include the name with an empty value. Do not drop the field, and do not substitute the text "null" — either produces a different hash. Sort by raw byte order, not a locale-aware comparison, or underscores and letters may order differently from Katana's. Hash the UTF-8 bytes.

Hash whatever arrives, not a fixed list. Katana may add fields to the callback. Build your verifier to sort and join *every* field present except `HASH`, and ignore fields you do not recognise in your business logic. A verifier hard-coded to today's nine fields will start failing the day a tenth is added.

6.2. Secret key

The Salt is the essential element in generating the hash. It is issued with your Key from **Integration** → **Generate Key + Salt** in your Katana portal, and is displayed **once**.

Credential Handling

Key	Identifies your account. Sent with every request.
Salt	Secret. Server-side only — never in browser code, a mobile application, or a repository. Signs your requests and verifies Katana's callbacks.

One active pair exists per merchant account. Regenerating **immediately invalidates** the previous pair, so deploy the new Salt before regenerating in production.

The Salt is the only thing protecting order creation. There is no IP allowlist and no second factor on the API. Treat Salt disclosure as a full compromise of your integration: regenerate immediately and reconcile every order created in the interim.

6.3. Code integration examples

PHP — CREATE ORDER (SCHEME 2, HMAC-SHA256)

```
$key="mk_9f2c"; $salt="your_salt";
$o = ["txnid"=>"UK108-ESN78452","amount"=>"100","productinfo"=>"Order
1001","email"=>"david@example.com"];
$msg = $o["txnid"]."|" . $o["amount"]."|" . $o["productinfo"]."|" . $o["email"];
$o["hash"] = hash_hmac("sha256", $msg, $key.$salt);
$o["key"]=$key; $o["mode"]="INTENT";
$o["return_url"]="https://yoursite.com/return";
$o["notify_url"]="https://yoursite.com/callback";
$ch=curl_init("https://katanapay.co/api/v1/katana-pay/order");
curl_setopt_array($ch,[CURLOPT_POST=>1,CURLOPT_RETURNTRANSFER=>1,
    CURLOPT_HTTPHEADER=>["Content-Type: application/json"],CURLOPT_POSTFIELDS=>json_encode($o)]);
$res=json_decode(curl_exec($ch),true);
if (!empty($res["reused"])) { /* collision – do NOT redirect; see 5.1 */ }
header("Location: ".$res["pay_url"]);
```

PHP — CREATE ORDER (SCHEME 1, SHA-512)

```
$seq = implode("|", [$key,$txnid,$amount,$productinfo,$firstname,$email,
    "", "", "", "", "", "", "", "", "", "", $salt]);
$hash = hash("sha512", $seq); // lowercase hex
```

NODE.JS — CREATE ORDER (SCHEME 2)

```
import crypto from "crypto";
const key="mk_9f2c", salt="your_salt";
const o={ txnid:"UK108-ESN78452", amount:"100", productinfo:"Order 1001",
email:"david@example.com" };
const msg=[o.txnid,o.amount,o.productinfo,o.email].join("|");
o.hash=crypto.createHmac("sha256",key+salt).update(msg).digest("hex");
const r=await fetch("https://katanapay.co/api/v1/katana-pay/order",{method:"POST",
  headers:{"Content-Type":"application/json"},
  body:JSON.stringify({...o,key,mode:"INTENT",
    return_url:"https://yoursite.com/return",
    notify_url:"https://yoursite.com/callback"}}));
const data=await r.json();
if (data.reused) { /* collision – do NOT redirect; see 5.1 */ }
// redirect the customer to data.pay_url
```

NODE.JS — CREATE ORDER (SCHEME 1)

```
const seq = [key,txnid,amount,productinfo,firstname,email,
  "", "", "", "", "", "", "", "", "", salt].join("|");
const hash = crypto.createHash("sha512").update(seq,"utf8").digest("hex");
```

PYTHON — CREATE ORDER (SCHEME 2)

```
import hmac,hashlib,requests
key,salt="mk_9f2c","your_salt"
o={"txnid":"UK108-ESN78452","amount":"100","productinfo":"Order 1001","email":"david@example.com"}
msg="|".join([o["txnid"],o["amount"],o["productinfo"],o["email"]])
o["hash"]=hmac.new((key+salt).encode(),msg.encode(),hashlib.sha256).hexdigest()
o.update(key=key, mode="INTENT",
  return_url="https://yoursite.com/return",
  notify_url="https://yoursite.com/callback")
data=requests.post("https://katanapay.co/api/v1/katana-pay/order",json=o).json()
if data.get("reused"): pass # collision – do NOT redirect; see 5.1
# redirect the customer to data["pay_url"]
```

PYTHON — CREATE ORDER (SCHEME 1)

```
seq = "|".join([key,txnid,amount,productinfo,firstname,email] + [""]*10 + [salt])
hash = hashlib.sha512(seq.encode("utf-8")).hexdigest()
```


VERIFYING A CALLBACK — NODE.JS

```
function verify(body, salt){
  const got = String(body.HASH ?? "");
  const f = { ...body }; delete f.HASH;
  const str = Object.keys(f).sort()
    .map(k => k + "=" + (f[k] == null ? "" : f[k])).join("~") + salt;
  const exp = crypto.createHash("sha256").update(str,"utf8").digest("hex").toUpperCase();
  return exp.length === got.length &&
    crypto.timingSafeEqual(Buffer.from(exp), Buffer.from(got));
}
```

VERIFYING A CALLBACK — PHP

```
function verify($body,$salt){
  $got=$body["HASH"] ?? ""; unset($body["HASH"]); ksort($body);
  $parts=[]; foreach($body as $k=>$v){$parts[]=$k."=".$v."?";}
  return hash_equals(strtoupper(hash("sha256", implode("~",$parts).$salt)), $got);
}
```

VERIFYING A CALLBACK — PYTHON

```
import hashlib, hmac
def verify(body, salt):
    got = body.get("HASH","")
    f = {k:v for k,v in body.items() if k != "HASH"}
    s = "~".join(f"{k}={v} " if f[k] is None else f"{k}={v}" for k in sorted(f)) + salt
    exp = hashlib.sha256(s.encode("utf-8")).hexdigest().upper()
    return hmac.compare_digest(exp, got)
```

7. STATUS ENQUIRY

Use this API to verify transaction status **before service delivery**. It is the authoritative record: the callback is a convenience that can be missed (§8.3), the enquiry always reflects the truth.

Method	Environment	URL
GET	Production	https://katanapay.co/api/pay-status/{katana_order_id}

Where `{katana_order_id}` is the `order.id` UUID returned when the order was created — **not** your `txnid` .

The enquiry uses a different status vocabulary from the callback. It returns a lowercase `status` field and **no** `RESPONSE_CODE` at all. Match on `status` and `terminal` :

Enquiry <code>status</code>	<code>terminal</code>	Equivalent callback	Deliver?
SUCCESS	true	Captured / 000	Yes
PENDING	false	Pending / 005	No — still open
FAILED	true	Failed / 004	No — but see §7.3
EXPIRED	true	Expired / 003	No — but see §7.3

Deliver only on `status = "SUCCESS"` . Compare case-insensitively and treat any value not in this table as not-paid.

7.1. Status Enquiry Request

```
GET https://katanapay.co/api/pay-status/8d20c0b6-1d0f-4389-bc0b-d1d9d02bfed6
```

LONG-POLL VARIANT

Add `?wait=1` to hold the request open until the order reaches a terminal state, up to approximately 25 seconds. This lets a payment page show the result within about half a second of the credit landing, without polling in a tight loop. On timeout the request returns the current, non-terminal payload — it does **not** error, so do not write a `catch` that treats a normal timeout as failure.

```
GET https://katanapay.co/api/pay-status/8d20c0b6-...?wait=1
```

Never use `?wait=1` in a batch sweep. Each request holds a connection for up to 25 seconds — a 500-order reconciliation run would hold 500 connections for 25 seconds each. Use the plain form for sweeps and the long-poll form only where a person is watching a screen.

The enquiry is unauthenticated and its response is unsigned. The order UUID is the capability. It provides no cryptographic assurance beyond TLS — for that, verify the callback `HASH` (§6.1 B).

7.2. Status Enquiry Response

```
{
  "order_id": "UK108-ESN78452",
  "amount": 100,
  "currency_code": "INR",
  "status": "SUCCESS",
  "terminal": true,
  "rrn": "123456789101",
  "mode": "INTENT",
  "proof_submitted": false,
  "return_url": "https://yoursite.com/return"
}
```

Field	Description
status	Current status, in the enquiry vocabulary above.
terminal	true once the order reached a final state. SUCCESS is permanent; EXPIRED and FAILED are provisional (§7.3).
rrn	Bank reference for the credit, when one was captured. See the note below.
proof_submitted	true when the payer uploaded a payment proof; the order is then held for operator review (§4.4).

RRN is the acquirer reference when one was captured. Where a payment is confirmed without a bank reference being supplied, Katana generates a deterministic internal reference in the same 12-digit shape. Such a value **will not appear on a bank statement**. If a reconciliation lookup by RRN finds nothing at the bank, raise it with Katana rather than assuming the payment is missing.

The order id is an unguessable capability, shared with the payer by design. It appears in the `pay_url` you send the customer to, so it is not a secret between you and Katana — but anyone holding it can read that order's amount, status and RRN. Keep it out of email subjects, shareable pages, client-side analytics, ad-network URL parameters and third-party logs. Address your own systems by your `ORDER_ID`.

7.3. Expiry and late credits

Expired and Failed are not permanent. If the customer's payment lands after the order expired, or goes through after an earlier attempt failed, Katana honours it and the order becomes SUCCESS — because the customer really did pay. Katana then sends a **second callback** with `STATUS = "Captured"` (§8.3). Your handler must let it overwrite the earlier Expired or Failed, not discard it as a duplicate.

Practical consequences you must build for:

- Before you cancel an order, void a reservation or release stock on `Expired` , **re-run the status enquiry**.
- Your reconciliation sweep (§10.2) must re-check every order that ended `EXPIRED` in the last few days, not just the ones still `PENDING` .
- Treat `SUCCESS` as permanent; treat `EXPIRED` and `FAILED` as "probably not paid, verify before acting irreversibly".

8. MERCHANT CALLBACKS

Use the callback to be notified when the payment flow reaches a terminal status. Share your callback URL with the Katana team and it will be configured for you, or pass `notify_url` per order to override it.

An issued Key + Salt is a prerequisite for receiving any callback. Katana never sends an unsigned callback, so an account with no signing credentials receives nothing at all — the delivery is recorded as skipped on Katana's side. Confirm your credentials are issued before testing your endpoint.

8.1. Call-back Response

```
{
  "PAY_ID": "9323643809301818",
  "ORDER_ID": "UK108-ESN78452",
  "TXN_ID": "4694e5ce-b8cd-475a-82b9-eec7bb846367",
  "AMOUNT": "100",
  "CURRENCY_CODE": "356",
  "STATUS": "Captured",
  "RESPONSE_CODE": "000",
  "RRN": "123456789101",
  "RESPONSE_DATE_TIME": "2026-09-08T11:35:42.991Z",
  "HASH": "8EAC6B6DC198A6B3DCA18995B0843A2A9D305825906261E0E34B425C7329DEBC"
}
```

The body has the same fields for every outcome. A failed and an expired order, and the second callback of an order that is paid after it expired (§7.3):

```
// Failed: the payment was declined. RRN is an empty string.
{ "ORDER_ID": "UK108-ESN78453", "STATUS": "Failed", "RESPONSE_CODE": "004", "RRN": "", ... }

// Expired: no payment arrived in time. RRN is an empty string.
{ "ORDER_ID": "UK108-ESN78454", "STATUS": "Expired", "RESPONSE_CODE": "003", "RRN": "", ... }

// The same order, paid after it expired: a SECOND callback, some time later
{ "ORDER_ID": "UK108-ESN78454", "STATUS": "Captured", "RESPONSE_CODE": "000", "RRN":
"123456789102", ... }
```

Field name	Description	Example value
PAY_ID	Payment identifier assigned by Katana	9323643809301818
ORDER_ID	Your merchant reference number	UK108-ESN78452
TXN_ID	Transaction identifier generated by Katana	4694e5ce-...
AMOUNT	Amount as booked on the order , normalised (see below)	100
CURRENCY_CODE	3-digit ISO 4217 numeric code	356
STATUS	Transaction status	Captured
RESPONSE_CODE	Code for transaction status	000
RRN	Bank reference number. Empty string when none was captured.	123456789101
RESPONSE_DATE_TIME	Time the callback payload was first built. Not refreshed on retries.	2026-09-08T11:35:42.991Z
HASH	Value generated by SHA-256 (§6.1 B)	8EAC6B6D...

Verify the HASH before acting, then re-check the amount. An unverified callback is indistinguishable from a forgery — anyone can POST to your URL. Confirm the signature (§6.1 B), then confirm the amount matches your pending order, before delivering anything.

Compare AMOUNT numerically, never as a string. Katana normalises it: ₹100.50 is sent as "100.5" , ₹100.00 as "100" . A string comparison against the amount you booked fails on any order with a paise value.

```
// WRONG - fails for 100.50
if (body.AMOUNT !== order.amount) reject();

// RIGHT
if (Math.round(Number(body.AMOUNT) * 100) !== order.amountMinor) reject();
```

Verify the HASH over the values **exactly as they arrived** — do not reformat AMOUNT before hashing.

AMOUNT is the amount **booked on the order**, echoed back. It is not independently derived from the credit, so it does not by itself prove the customer paid that amount. Reconcile against your bank statement by RRN (§10.2) for that assurance.

CALLBACK HEADERS

Header	Meaning
x-event-type	payin.status
x-attempt	Delivery attempt number, from 1 . Anything above 1 is a retry — you have very likely already processed this order.
x-timestamp	Unix seconds at send.
x-payload-hash	Digest of the canonical payload. For logging only.
x-signature	Reserved for Katana's internal transport. Do not use it for authentication. Verify the HASH field in the body (§6.1 B) — that is the only signature bound to your Salt.

8.2. Merchant-Response

Reply HTTP 200 . A body is optional; the following is accepted and recommended:

```
{ "STATUS": "TRUE" }
```

Reply within 8 seconds. Acknowledge first, process afterwards: write the callback to a queue or table, return `200` immediately, and do your fulfilment work asynchronously. A handler that charges a card, sends an email or waits on a third party before replying will blow the deadline, be recorded as failed, and be retried — which is how merchants end up double-fulfilling.

Handle callbacks idempotently. Key your handler on `ORDER_ID + STATUS` and make a repeat of the same status a no-op that still returns `200`. A `Captured` after an `Expired` or `Failed` is **not** a repeat (§7.3). That dedupe is what protects you from replay — make it durable, not in-memory.

`RESPONSE_DATE_TIME` is the first-attempt time and is not refreshed per delivery, so it cannot be used as a freshness bound.

8.3. Delivery and retries

Callbacks per order	One at the first terminal status, and a second (<code>Captured</code>) if an <code>Expired</code> or <code>Failed</code> order is paid afterwards
Attempts	7 : the first at once, then one after each wait below
Backoff	1 min → 5 min → 15 min → 1 hour → 6 hours → 24 hours
Your response deadline	8 seconds per attempt; a slower reply is recorded as a failure and retried
Success	Any <code>2xx</code>
After 7 failures	The callback is abandoned . Katana does not deliver it later.
<code>notify_url</code> requirements	http(s), publicly resolvable; must not resolve to a private, loopback or link-local address

Every attempt is recorded on Katana's side. In the portal, **Orders** shows each attempt for an order with what your server answered, **Resend** sends a callback again, and **Send test event** sends a sample of each status to your callback URL.

The callback is a convenience, not a guarantee. Each one is abandoned after roughly 31 hours of failures. An `Expired` or `Failed` order that is paid afterwards (§7.3) gets a second, `Captured` callback, with the same delivery rules. The status enquiry (§7) is the authoritative record, and the daily reconciliation sweep (§10.2) is mandatory, not optional.

9. GATEWAY OPTIONS AND RESPONSE

9.1. Response Code for Valid Transaction

TABLE 4 — RESPONSE CODES FOR VALID TRANSACTIONS

RESPONSE_CODE	STATUS	Terminal	Meaning
000	Captured	yes	Paid. Deliver the goods or service.
005	Pending	no	Awaiting payment.
004	Failed	soft ¹	Declined or failed.
003	Expired	soft ¹	Katana stopped waiting for the credit.

¹ **Expired** and **Failed** are not permanent. A payment that goes through afterwards moves the order to **SUCCESS** , and a second callback with **Captured** is sent — see §7.3.

Treat any unrecognised code as not-paid. Katana currently emits only the four codes above. Codes in the `0xx` range may be added; a code you do not recognise must never be read as success — log it and treat the order as unpaid.

Captured is not the same as settled. A `000` callback means the payment was **collected**. Money reaching your bank account follows on the settlement cycle — see §10.3.

9.2. Response Code for Invalid Transaction

TABLE 5 — ERRORS AND INDETERMINATE OUTCOMES ON ORDER CREATION

HTTP	Error	Meaning and action
200	<code>none</code> — <code>reused: true</code>	Not an error: this <code>txnid</code> already exists. Check <code>order.amount</code> matches what you sent; if not, you have collided with another order (§5.1).
401	<code>invalid key</code>	The Key is not recognised. Check you are using the right environment's credentials.
401	<code>signature mismatch</code>	The hash did not match. Work the debugging list below.
403	<code>code</code> <code>MERCHANT_BLOCKED</code> / <code>MERCHANT_SUSPENDED</code>	The merchant account is blocked or suspended. Contact your Katana account manager.

403	<code>code</code> <code>PAYIN_NOT_ENABLED</code> / <code>PAYOUT_NOT_ENABLED</code>	The account is not set up for this service: a pay-out only account takes no pay-in orders, a pay-in only account sends no payouts. Contact your Katana account manager.
422	<code>code</code> <code>AMOUNT_BELOW_MIN</code> , <code>AMOUNT_ABOVE_MAX</code> , <code>UPI_LIMIT_EXCEEDED</code> , <code>DAILY_LIMIT_EXCEEDED</code>	The order is outside the account's limits; no order was created. The body carries <code>field</code> , <code>limit</code> and <code>actual</code> . Switch on <code>code</code> , not on the message. A repeated <code>txnid</code> is never limited.
429	<code>code</code> <code>RATE_LIMITED</code>	Too many orders in one second. Retry with the same <code>txnid</code> after the <code>Retry-After</code> header (1 second).
400	<code>invalid amount</code>	The amount is missing, zero, negative or not a number.
400	validation error	A field failed validation — for example a <code>return_url</code> that is not http(s), or more than 30 entries in <code>receiver_vpas</code> . The message text is diagnostic; do not string-match on it .
404	—	Status enquiry only: unknown order id. Check you sent the <code>order.id</code> UUID, not your <code>txnid</code> .
500	<code>order create failed</code>	Indeterminate, not failed. Retry with the same <code>txnid</code> .
—	timeout / connection reset	Indeterminate. Retry with the same <code>txnid</code> , then fall back to a status enquiry. Never re-issue with a new <code>txnid</code> .

RETRY GUIDANCE

Treat `5xx` and network timeouts as **indeterminate, not failed**. Retry with the **same** `txnid` : order creation is idempotent on that value, so a retry returns the original order rather than creating a second one. Back off between attempts rather than retrying in a tight loop, give up after a small number of attempts, and fall back to a status enquiry (§7).

DEBUGGING 401 SIGNATURE MISMATCH

This is the most common integration failure. Work down the list in order — the cause is nearly always one of the first three.

1. **Wrong scheme.** Confirm whether your Key is Scheme 1 (SHA-512) or Scheme 2 (HMAC-SHA256) — §6.1. Signing with the other one fails 100% of the time.
2. **The amount changed between signing and sending.** You signed `100` and sent `100.00` , or a JSON library re-serialised the string as a number. Build the string once, sign it, send it (§5.2).
3. **Wrong separator count** (Scheme 1). Eleven `|` between `email` and `salt` . Print the signing string and count them.
4. **A field in the hash but not the request, or the reverse.** Omitted fields sign as empty strings in position; they do not disappear.

5. **Encoding.** Hash the UTF-8 bytes. Non-ASCII in `productinfo` or `firstname` is the usual culprit.
6. **Stale credentials.** Regenerating a Key + Salt invalidates the previous pair immediately (§6.2). Confirm the deployed Salt is the current one.

Still stuck: send Katana your `txnid` , your signing string **with the Salt redacted**, and the hash you computed.

Keep signed free-text fields simple. Avoid `|` , `~` and `=` in `txnid` and `productinfo` — they are the separators used by the two hashing constructions, and are not escaped. A safe `txnid` pattern is `^[A-Za-z0-9_-]{1,60}$` .

9.3. Payment method codes

TABLE 6 — SUPPORTED PAYMENT METHODS

Payment method	mode value	Notes
UPI — Intent deeplink	INTENT	Returns <code>upi_intent</code> and per-application deeplinks.
UPI — QR	QR	Returns <code>qr_payload</code> . The default.

9.4. Supported Currency and Currency Code

TABLE 7 — SUPPORTED CURRENCY WITH CODES

Name	Abbreviation	Code
Indian Rupee	INR	356

Requests use the alphabetic code (`currency: "INR"`); callbacks return the ISO 4217 numeric code (`CURRENCY_CODE: "356"`).

Only INR is supported. Send `INR` or omit the field. Another currency value is not currently rejected at the API, but the payment instrument is always denominated in rupees — sending anything else will produce an inconsistent order.

10. GOING LIVE AND DAY-2 OPERATIONS

10.1. Go-live checklist

- Production **Key + Salt** issued; Salt stored server-side only — never in browser code, an app binary, or a repository.

- **Signing scheme confirmed** (§6.1) and a signed order accepted end to end.
- `txnid` scheme is **globally unique** and prefixed with your merchant code (§5.1).
- Your response handler **checks** `reused` and compares `order.amount` (§5.1).
- Amount is built as a string once, signed, and sent unchanged (§5.2).
- **Collection UPI ID confirmed** on your account, and verified on a real test payment (§5.1).
- Default `return_url` and callback URL saved with Katana, or passed per order.
- Callback handler **verifies** `HASH` before acting (§6.1 B), compares `AMOUNT` **numerically** (§8.1), and returns `200` **within 8 seconds**.
- Callback handler is **idempotent** on `ORDER_ID`, with durable dedupe.
- Browser-return parameters are treated as **display only** (§4.5).
- Reconciliation sweep runs on a schedule and can carry an order for **days** (held orders never expire, §4.4).
- `EXPIRED` and `FAILED` orders are **re-checked** before being written off, and a later `Captured` callback is accepted (§7.3).
- A test payment completed in each of: **success, failure, expiry, and callback-not-received**.
- Escalation contact and account manager recorded on both sides (§11).

KATANA'S PRE-GO-LIVE REVIEW

Per §5.3, Katana reviews the merchant integration before enabling production. The review covers: signature verification present on the callback handler; amount and order-reference comparison present; idempotent handling; and a working reconciliation sweep. Have these ready.

10.2. Daily reconciliation

Run a sweep at least once a day. It must be able to close orders the callback never closed — given that the callback is sent once and abandoned after ~31 hours of failures (§8.3), and that held orders can stay open for days (§4.4), this is the safety net the rest of the design depends on.

1. Select your own orders that are not in a final state, plus any that reached a final state in the last 7 days.
2. For each, `GET /api/pay-status/{katana_order_id}` — **without** `?wait=1` (§7.1).
3. Apply the status table in §7. **Re-check every** `EXPIRED` and `FAILED` order at least once (§7.3).
4. Compare amounts numerically and investigate any mismatch before adjusting your books.
5. Download the day's transactions from **Transactions** → **Export** in your Katana portal and match on `RRN` against your bank statement. The export carries the date, your reference, amount, status, payment id, bank reference and UPI handle.

WHAT TO ESCALATE

- Any order Katana reports `SUCCESS` that you have no record of.
- Any order you fulfilled that Katana does not report `SUCCESS`.

- Any `RRN` appearing against two of your orders.
- Any `RRN` that your bank statement does not show (§7.2).

Keep `order.id` , `ORDER_ID` , `RRN` and the raw callback body for every transaction — those four are what every Katana investigation asks for.

10.3. Settlement DECISION

Item	Value
Settlement cycle	DECISION
Cut-off time	DECISION
Settlement account	The bank account or UPI ID registered at onboarding. Changing it requires a signed request and re-verification.
Minimum settlement	DECISION
Fees / MDR	DECISION — per your commercial agreement; not exposed in the API
Statement location	DECISION

A callback with `STATUS: "Captured"` means the payment was **collected**. It does not mean it has been **settled** to your account. Settlement follows on the cycle above. Confirm every value here with your account manager before go-live.

10.4. Refunds and disputes DECISION

Katana does not currently expose a refund API. A captured payment cannot be reversed through the integration.

Situation	Process
Refund to a customer	Raise with your Katana account manager, quoting <code>ORDER_ID</code> and <code>RRN</code> . Turnaround: DECISION
Customer says they paid but the order shows unpaid	Run a status enquiry first (§7), and re-check if the order shows <code>EXPIRED</code> (§7.3). If still unpaid, raise it with the UTR from the customer's bank app.
Duplicate payment	As for a refund. Katana will not have auto-reversed it.

Design your checkout on the assumption that a captured payment is final. If your business needs customer-initiated reversals, raise it before go-live.

11. SUPPORT AND ESCALATION DECISION

Severity	Examples	Channel	Response
P1 — payments down	Orders failing platform-wide; no callbacks arriving at all	DECISION	DECISION
P2 — money at risk	A payment collected but not reflected; a duplicate <code>RRN</code> ; an expired order that took a payment	DECISION	DECISION
P3 — integration	<code>signature mismatch</code> in testing; field questions	DECISION	DECISION

INCLUDE IN EVERY TICKET

A ticket without these will be returned:

- `ORDER_ID` (your reference) **and** `order.id` (Katana's UUID)
- `RRN` , if the payment reached a bank
- The exact request body you sent, including the `hash` — **never the Salt**
- The raw callback body you received, if any
- Timestamps in UTC, with the timezone stated

Katana Payment Gateway Integration Guide · Version 1.1 · For integration support, contact your Katana account manager.